11
Dynamic Programming

As algorithm designers and programmers, we are often charged with building a program to find the *best* solution for all problem instances. It is usually easy to write a program which gives a decent and correct solution, but ensuring that it always returns the absolute best solution requires us to think deeply about the problem.

Dynamic programming is a very powerful, general tool for solving optimization problems on left-right-ordered items such as character strings. Once understood it is relatively easy to apply, but many people have a difficult time understanding it.

Dynamic programming looks like magic until you have seen enough examples. Start by reviewing our binomial coefficient function in Chapter 6, as an example of how we stored partial results to help us compute what we were looking for. Then review Floyd's all-pairs shortest-path algorithm in Section 10.3.2. Only then should you study the two problems in the sections below. The first is a classic example of dynamic programming which appears in every textbook. The second is a more ad hoc example representative of using dynamic programming to design new algorithms.

11.1 Don't Be Greedy

Many problems call for finding the best solution satisfying certain constraints. We have a few tricks available to tackle such jobs. For example, the backtracking problems of Chapter 8 often asked for the largest, smallest, or highest-scoring configuration. Backtracking searches all possible solutions and selects the best one, and hence *must* return the correct answer. But this approach is only feasible for small problem instances.

Correct and efficient algorithms are known for many important graph problems, including shortest paths, minimum spanning trees, and matchings, as discussed in Chapter 10. Always be on the lookout for instances of these problems, so you can just plug in the appropriate solution.

Greedy algorithms focus on making the best local choice at each decision point. For example, a natural way to compute a shortest path from x to y might be to walk out of x, repeatedly following the cheapest edge until we get to y. Natural, but wrong! Indeed, in the absence of a correctness proof such greedy algorithms are very likely to fail.

So what can we do? Dynamic programming gives us a way to design custom algorithms which systematically search all possibilities (thus guaranteeing correctness) while storing results to avoid recomputing (thus providing efficiency).

Dynamic programming algorithms are defined by recursive algorithms/functions that describe the solution to the entire problem in terms of solutions to smaller problems. Backtracking is one such recursive procedure we have seen, as is depth-first search in graphs.

Efficiency in any such recursive algorithm requires storing enough information to avoid repeating computations we have done before. Why is depth-first search in graphs efficient? It is because we mark the vertices we have visited so we don't visit them again. Why is raw backtracking computationally expensive? Because it searches all possible paths/solutions instead of just the ones we haven't seen before.

Dynamic programming is a technique for efficiently implementing a recursive algorithm by storing partial results. The trick is to see that the naive recursive algorithm repeatedly computes the same subproblems over and over and over again. If so, storing the answers to them in a table instead of recomputing can lead to an efficient algorithm. To understand the examples which follow, it will help first to hunt for some kind of recursive algorithm. Only once you have a correct algorithm can you worry about speeding it up by using a results matrix.

11.2 Edit Distance

The problem of searching for patterns in text strings is of unquestionable importance. Indeed, we presented algorithms for string search in Chapter 3. However, there we limited discussion to *exact* string matching, finding where the pattern string s was exactly contained in the text string t. Life is often not that simple. Misspellings in either the text or pattern rob of us exact similarity. Evolutionary changes in genomic sequences or language usage imply that we often search with archaic patterns in mind: "Thou shalt not kill" morphs into "You should not murder."

If we are to deal with inexact string matching, we must first define a cost function telling us how far apart two strings are, i.e., a distance measure between pairs of strings. A reasonable distance measure minimizes the cost of the *changes* which have to be made to convert one string to another. There are three natural types of changes:

- *Substitution* — Change a single character from pattern s to a different character in text t, such as changing "shot" to "spot".

- *Insertion* — Insert a single character into pattern s to help it match text t, such as changing "ago" to "agog".
- *Deletion* — Delete a single character from pattern s to help it match text t, such as changing "hour" to "our".

Properly posing the question of string similarity requires us to set the cost of each of these string transform operations. Setting each operation to cost one step defines the *edit distance* between two strings. Other cost values also yield interesting results, as will be shown in Section 11.4.

But how can we compute the edit distance? We can define a recursive algorithm using the observation that the last character in the string must either be matched, substituted, inserted, or deleted. Chopping off the characters involved in the last edit operation leaves a pair of smaller strings. Let i and j be the last character of the relevant prefix of s and t, respectively. There are three pairs of shorter strings after the last operation, corresponding to the strings after a match/substitution, insertion, or deletion. *If* we knew the cost of editing the three pairs of smaller strings, we could decide which option leads to the best solution and choose that option accordingly. We *can* learn this cost, through the magic of recursion:

```
#define MATCH           0       /* enumerated type symbol for match */
#define INSERT          1       /* enumerated type symbol for insert */
#define DELETE          2       /* enumerated type symbol for delete */

int string_compare(char *s, char *t, int i, int j)
{
        int k;                  /* counter */
        int opt[3];             /* cost of the three options */
        int lowest_cost;        /* lowest cost */

        if (i == 0) return(j * indel(' '));
        if (j == 0) return(i * indel(' '));

        opt[MATCH] = string_compare(s,t,i-1,j-1) + match(s[i],t[j]);
        opt[INSERT] = string_compare(s,t,i,j-1) + indel(t[j]);
        opt[DELETE] = string_compare(s,t,i-1,j) + indel(s[i]);

        lowest_cost = opt[MATCH];
        for (k=INSERT; k<=DELETE; k++)
                if (opt[k] < lowest_cost) lowest_cost = opt[k];

        return( lowest_cost );
}
```

This program is absolutely correct – convince yourself. It is also impossibly slow. Running on our computer, it takes several seconds to compare two 11-character strings, and the computation disappears into never-never land on anything longer.

Why is the algorithm so slow? It takes exponential time because it recomputes values again and again and again. At every position in the string, the recursion branches three ways, meaning it grows at a rate of at least 3^n – indeed, even faster since most of the calls reduce only one of the two indices, not both of them.

So how can we make the algorithm practical? The important observation is that most of these recursive calls are computing things that have already been computed before. How do we know? Well, there can only be $|s| \cdot |t|$ possible unique recursive calls, since there are only that many distinct (i, j) pairs to serve as the parameters of recursive calls. By storing the values for each of these (i, j) pairs in a table, we can avoid recomputing them and just look them up as needed.

A table-based, dynamic programming implementation of this algorithm is given below. The table is a two-dimensional matrix m where each of the $|s| \cdot |t|$ cells contains the cost of the optimal solution of this subproblem, as well as a parent pointer explaining how we got to this location:

```
typedef struct {
        int cost;               /* cost of reaching this cell */
        int parent;             /* parent cell */
} cell;

cell m[MAXLEN+1][MAXLEN+1];     /* dynamic programming table */
```

The dynamic programming version has three differences from the recursive version. First, it gets its intermediate values using table lookup instead of recursive calls. Second, it updates the `parent` field of each cell, which will enable us to reconstruct the edit-sequence later. Third, it is instrumented using a more general `goal_cell()` function instead of just returning `m[|s|][|t|].cost`. This will enable us to apply this routine to a wider class of problems.

Be aware that we adhere to certain unusual string and index conventions in the following routines. In particular, we assume that each string has been padded with an initial blank character, so the first real character of string `s` sits in `s[1]`. This was done using the following input fragment:

```
s[0] = t[0] = ' ';
scanf("%s",&(s[1]));
scanf("%s",&(t[1]));
```

Why did we do this? It enables us to keep the matrix `m` indices in sync with those of the strings for clarity. Recall that we must dedicate the zeroth row and columns of `m` to store the boundary values matching the empty prefix. Alternatively, we could have left the input strings intact and just adjusted the indices accordingly.

```
int string_compare(char *s, char *t)
{
        int i,j,k;              /* counters */
        int opt[3];             /* cost of the three options */
```

```
        for (i=0; i<MAXLEN; i++) {
                row_init(i);
                column_init(i);
        }

        for (i=1; i<strlen(s); i++)
                for (j=1; j<strlen(t); j++) {
                        opt[MATCH] = m[i-1][j-1].cost + match(s[i],t[j]);
                        opt[INSERT] = m[i][j-1].cost + indel(t[j]);
                        opt[DELETE] = m[i-1][j].cost + indel(s[i]);

                        m[i][j].cost = opt[MATCH];
                        m[i][j].parent = MATCH;
                        for (k=INSERT; k<=DELETE; k++)
                                if (opt[k] < m[i][j].cost) {
                                        m[i][j].cost = opt[k];
                                        m[i][j].parent = k;
                                }
                }

        goal_cell(s,t,&i,&j);
        return( m[i][j].cost );
}
```

Notice the order that the cells of the matrix are filled in. To determine the value of cell (i, j) we need three values sitting and waiting for us, namely, the cells $(i-1, j-1)$, $(i, j-1)$, and $(i-1, j)$. Any evaluation order with this property will do, including the row-major order used in this program.

Below is an example run, showing the cost and parent values turning "thou shalt not" to "you should not" in five moves:

		y	o	u	-	s	h	o	u	l	d	-	n	o	t
:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
t:	1	1	2	3	4	5	6	7	8	9	10	11	12	13	13
h:	2	2	2	3	4	5	5	6	7	8	9	10	11	12	13
o:	3	3	2	3	4	5	6	5	6	7	8	9	10	11	12
u:	4	4	3	2	3	4	5	6	5	6	7	8	9	10	11
-:	5	5	4	3	2	3	4	5	6	6	7	7	8	9	10
s:	6	6	5	4	3	2	3	4	5	6	7	8	8	9	10
h:	7	7	6	5	4	3	2	3	4	5	6	7	8	9	10
a:	8	8	7	6	5	4	3	3	4	5	6	7	8	9	10
l:	9	9	8	7	6	5	4	4	4	4	5	6	7	8	9
t:	10	10	9	8	7	6	5	5	5	5	5	6	7	8	8
-:	11	11	10	9	8	7	6	6	6	6	6	5	6	7	8
n:	12	12	11	10	9	8	7	7	7	7	7	6	5	6	7
o:	13	13	12	11	10	9	8	7	8	8	8	7	6	5	6
t:	14	14	13	12	11	10	9	8	8	9	9	8	7	6	5

		y	o	u	-	s	h	o	u	l	d	-	n	o	t
:	-1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
t:	2	0	0	0	0	0	0	0	0	0	0	0	0	0	0
h:	2	0	0	0	0	0	0	0	1	1	1	1	1	1	1
o:	2	0	0	0	0	0	0	0	1	1	1	1	1	0	1
u:	2	0	2	0	1	1	1	1	0	1	1	1	1	1	1
-:	2	0	2	2	0	1	1	1	1	0	0	0	1	1	1
s:	2	0	2	2	2	0	1	1	1	1	0	0	0	0	0
h:	2	0	2	2	2	2	0	1	1	1	1	1	1	0	0
a:	2	0	2	2	2	2	2	0	0	0	0	0	0	0	0
l:	2	0	2	2	2	2	2	0	0	0	1	1	1	1	1
t:	2	0	2	2	2	2	2	0	0	0	0	0	0	0	0
-:	2	0	2	2	0	2	2	0	0	0	0	0	1	1	1
n:	2	0	2	2	2	2	2	0	0	0	0	2	0	1	1
o:	2	0	0	2	2	2	2	0	0	0	0	2	2	0	1
t:	2	0	2	2	2	2	2	2	0	0	0	2	2	2	0

11.3 Reconstructing the Path

The dynamic programming implementation above returns the cost of the optimal solution, but not the solution itself. Knowing you can convert "thou shalt not" to "you should not" in five moves is fine and dandy, but what is the sequence of editing operations that does it?

The possible solutions to a given dynamic programming problem are described by paths through the dynamic programming matrix, starting from the initial configuration (the pair of empty strings $(0, 0)$) down to the final goal state (the pair of full strings $(|s|, |t|)$). The key to building the solution is to reconstruct the decisions made at every step along the path that lead to the optimal cost. These decisions have been recorded in the `parent` field of each array cell.

Reconstructing these decisions is done by walking backward from the goal state, following the `parent` pointer to an earlier cell. We repeat this process until we arrive back at the initial cell. The `parent` field for `m[i,j]` tells us whether the transform at (i, j) was `MATCH`, `INSERT`, or `DELETE`. The edit sequence from "thou-shalt-not" to "you-should-not" is `DSMMMMMISMSMMMM` – meaning delete the first "t", replace the "h" with "y", match the next five characters before inserting an "o", replace "a" with "u", and finally replace the "t" with a "d".

Walking backward reconstructs the solution in reverse order. However, clever use of recursion can do the reversing for us:

```
reconstruct_path(char *s, char *t, int i, int j)
{
        if (m[i][j].parent == -1) return;

        if (m[i][j].parent == MATCH) {
                reconstruct_path(s,t,i-1,j-1);
                match_out(s, t, i, j);
                return;
        }
        if (m[i][j].parent == INSERT) {
                reconstruct_path(s,t,i,j-1);
                insert_out(t,j);
                return;
        }
        if (m[i][j].parent == DELETE) {
                reconstruct_path(s,t,i-1,j);
                delete_out(s,i);
                return;
        }
}
```

For many problems, including edit distance, the tour can be reconstructed without explicitly retaining the last-move pointer array. The trick is working backward to see what move took you to the current cell at the given cost.